

LLD Case Study 3: Movie Ticket Booking System (BookMyShow / Fandango)

Target Patterns: Concurrency Control, State Machine, Factory Pattern, Strategy Pattern

Key Engineering Focus: High-concurrency seat locking, race condition elimination, temporary seat reservations with TTL expiration, and payment processing.

1. Problem Statement & Functional Requirements

Design the core booking engine for a cinema platform. Thousands of concurrent users vie for high-demand premiere seats.

Requirements:

- Cinema Hierarchy:** Cinema $\rightarrow$ Audi / Screen $\rightarrow$ Show $\rightarrow$ Seats (Regular, VIP, Recliner).
- State Machine:** A seat can be `AVAILABLE`, `TEMPORARILY_LOCKED` (e.g., 5-minute payment hold window), or `BOOKED`.
- Concurrency Control:** Absolutely no two users can book or lock the same seat simultaneously. Handled via thread-safe atomic mutexes.
- Auto-Expiration:** If payment is not completed within the hold window, the lock automatically expires and the seat returns to `AVAILABLE`.
- Payment Processing:** Decoupled via Strategy Pattern (UPI, CreditCard, NetBanking).

2. Architecture & State Diagram

stateDiagram-v2

[*] $\rightarrow$ AVAILABLE

AVAILABLE $\rightarrow$ LOCKED : User selects seat (Acquires seat lock, starts 5-min TTL)

LOCKED $\rightarrow$ BOOKED : Payment Confirmed (Permanent state)

LOCKED $\rightarrow$ AVAILABLE : TTL Expired OR User Aborted Checkout

```
classDiagram
    class SeatStatus {
        <<enumeration>>
        AVAILABLE
        LOCKED
        BOOKED
    }

    class Seat {
        +String seatId
        +String row
        +int number
        +SeatType type
        +SeatStatus status
        +String lockedByUserId
        +float lockedAtTimestamp
        +Lock mutex
    }

    class BookingService {
        -Map~String, Seat~ seatRegistry
        +lockSeats(List~String~ seatIds, String userId) bool
        +confirmBooking(List~String~ seatIds, String userId, PaymentStrategy payment) Bool
        +releaseExpiredLocks() void
    }

    class PaymentStrategy {
        <<interface>>
        +pay(float amount) bool
    }

    BookingService ..> Seat : Locks & Mutates
    BookingService o-- PaymentStrategy : Payment Processing
```

3. Production-Grade Python Implementation

```
import threading
import time
from enum import Enum, auto
from abc import ABC, abstractmethod
from typing import List, Dict, Optional

# --- Enums & Seat Model ---
class SeatStatus(Enum):
    AVAILABLE = auto()
    LOCKED = auto()
    BOOKED = auto()

class Seat:
    def __init__(self, seat_id: str, price: float):
        self.seat_id = seat_id
        self.price = price
        self.status = SeatStatus.AVAILABLE
        self.locked_by: Optional[str] = None
        self.locked_at: float = 0.0
        self.lock = threading.Lock() # Fine-grained per-seat lock!

    def is_lock_expired(self, ttl_seconds: float) → bool:
        if self.status == SeatStatus.LOCKED and (time.monotonic() - self.locked_at) > ttl:
            return True
        return False

# --- Payment Strategy Pattern ---
class PaymentStrategy(ABC):
    @abstractmethod
    def process_payment(self, amount: float) → bool: pass

class CreditCardPayment(PaymentStrategy):
    def process_payment(self, amount: float) → bool:
        print(f"[Payment Gateway] Successfully charged ${amount:.2f} via Credit Card.")
        return True

class UPIPayment(PaymentStrategy):
    def process_payment(self, amount: float) → bool:
        print(f"[Payment Gateway] Successfully transferred ${amount:.2f} via UPI.")
```

```

        return True

# --- The Booking Engine ---
class BookingService:
    LOCK_TTL_SECONDS = 5.0 # For demo; in prod usually 300s (5 mins)

    def __init__(self, seats: List[Seat]):
        self.seats: Dict[str, Seat] = {s.seat_id: s for s in seats}
        self._global_lock = threading.Lock()

    def lock_seats(self, seat_ids: List[str], user_id: str) → bool:
        """
        Attempts to lock all requested seats atomically.
        Sorts seat_ids to guarantee strict global lock ordering (Deadlock Prevention!).
        """
        sorted_ids = sorted(seat_ids)
        acquired_locks = []

        try:
            # 1. Acquire all individual seat locks in order
            for sid in sorted_ids:
                seat = self.seats[sid]
                seat.lock.acquire()
                acquired_locks.append(seat.lock)

            # 2. Check if any seat is already booked or actively locked
            now = time.monotonic()
            for sid in sorted_ids:
                seat = self.seats[sid]
                # Auto-heal expired locks
                if seat.is_lock_expired(self.LOCK_TTL_SECONDS):
                    seat.status = SeatStatus.AVAILABLE
                    seat.locked_by = None

                if seat.status ≠ SeatStatus.AVAILABLE:
                    print(f"[Lock Rejected] Seat {sid} is not available (Status: {seat.st
                    return False

            # 3. Transition all seats to LOCKED
            for sid in sorted_ids:
                seat = self.seats[sid]
                seat.status = SeatStatus.LOCKED

```

```
        seat.locked_by = user_id
        seat.locked_at = now

    print(f"[Lock Success] Seats {seat_ids} temporarily locked for user {user_id}")
    return True

finally:
    # Release all locks so other queries can proceed
    for lock in reversed(acquired_locks):
        lock.release()

def confirm_booking(self, seat_ids: List[str], user_id: str, payment: PaymentStrategy)
    """Confirms booking upon successful payment capture."""
    sorted_ids = sorted(seat_ids)
    acquired_locks = []

    try:
        for sid in sorted_ids:
            seat = self.seats[sid]
            seat.lock.acquire()
            acquired_locks.append(seat.lock)

        # Verify ownership and valid lock
        for sid in sorted_ids:
            seat = self.seats[sid]
            if seat.status != SeatStatus.LOCKED or seat.locked_by != user_id:
                print(f"[Booking Failed] User {user_id} does not hold active lock for")
                return False

            if seat.is_lock_expired(self.LOCK_TTL_SECONDS):
                print(f"[Booking Failed] Lock on seat {sid} expired before checkout.")
                seat.status = SeatStatus.AVAILABLE
                return False

        total_amount = sum(self.seats[sid].price for sid in sorted_ids)

        # Execute payment
        if not payment.process_payment(total_amount):
            print("[Booking Failed] Payment gateway rejected transaction.")
            return False

        # Transition to permanently BOOKED
        for sid in sorted_ids:
```

```
        seat = self.seats[sid]
        seat.status = SeatStatus.BOOKED

    print(f"[Booking Confirmed] Seats {seat_ids} successfully booked for user {user_name}")
    return True

    finally:
        for lock in reversed(acquired_locks):
            lock.release()

# --- Concurrent Simulation Test ---
if __name__ == "__main__":
    seats_pool = [Seat("A1", 15.0), Seat("A2", 15.0)]
    booking_system = BookingService(seats_pool)

    def user_action(user_name: str, target_seat: str):
        success = booking_system.lock_seats([target_seat], user_name)
        if success:
            time.sleep(0.1) # Simulate checkout interaction
            booking_system.confirm_booking([target_seat], user_name, CreditCardPayment())

    # Two concurrent threads race for seat A1 simultaneously
    t1 = threading.Thread(target=user_action, args=("Alice", "A1"))
    t2 = threading.Thread(target=user_action, args=("Bob", "A1"))

    t1.start()
    t2.start()
    t1.join()
    t2.join()
```